

The Kuroko Paradigm: Augmenting Avatars in Augmented Reality with Reality Actuators

Émilie Fabre*
The University of Tokyo

Yuta Itoh†
The University of Tokyo

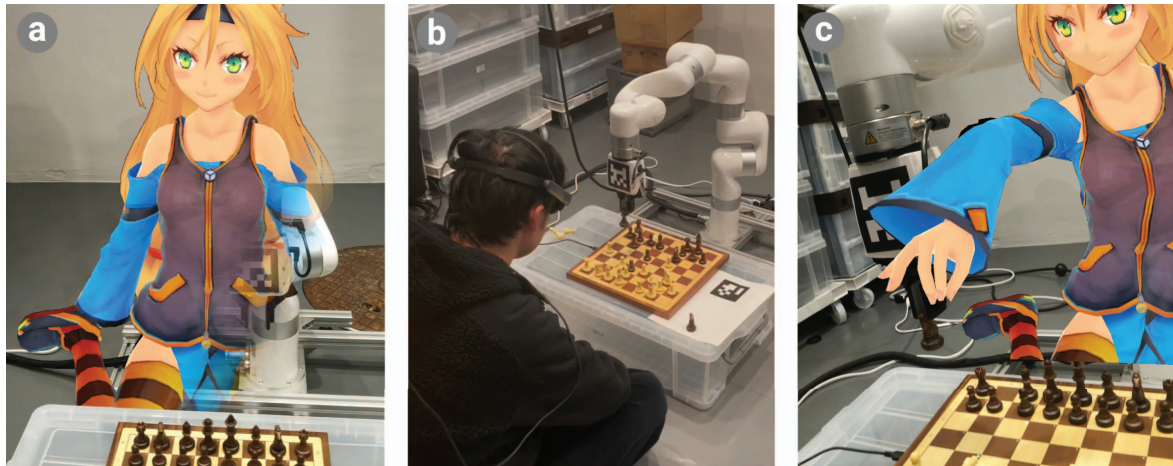


Figure 1: The Kuroko Chess system. a: Example picture showing the avatar gradually overlaying the robotic arm. b: A user wearing an HMD engaged in a game of chess against a robotic arm. c: User view of the avatar seemingly grabbing a chess piece (misalignment is due to the camera height being different than the user's eye height)

ABSTRACT

We present a system that enhances Virtual Avatars with Reality Actuators in order to allow them to interact with the real world. We demonstrate this idea by overlaying a 3D avatar onto a robotic arm engaged in a game of chess. We think that using Virtual Avatar to hide the action of reality actuators provides an improved experience through better presence, social presence, and engagement and allows for a more seamless interaction between real and virtual objects.

Index Terms: Human-centered computing—Interactive systems and tools; Human-centered computing—Mixed / augmented reality;

1 INTRODUCTION AND BACKGROUND

Research on avatar is currently active, both in the Human-Computer Interaction (HCI) [2, 12] and Artificial Intelligence (AI) [11, 13] field, with most research tending to focus on the rendering methods or style for avatars. However, in a world where Artificial Intelligence and AI are predicted to become pervasive [4, 9], few actually focus on methods to integrate them seamlessly in our world, a quality which has been shown to be positive for factors such as presence, engagement or immersion. [2, 12]

This is also true in the case of using avatars to represent virtual agents, i.e., an entity controlled by the program itself. [6] In a review on the convergence of AI, AR Agents (ARA), and the Internet of Things (IoT), Norouzi [9] mentions that these futuristic agents would become able to “physically influence throughout one’s house”. While Norouzi mostly focused on IoT, where agents usually interact

through smart switches or objects, ARA still lack the necessary physical presence for most applications.

In 2004, Aoki et al. [1] introduced a system that hinges close to this concept. By looking through a screen, you would see small avatars called “Kobito” seemingly pushing a self-moving cube. The “Kobito” project marks a step towards using a wide range of reality actuators in order to have avatars seamlessly interact in the real world. The now famous 2016 paper the “Wobbly Table” demonstrate the benefits of having such a seamless interaction between virtual and real objects. [7] By having a table move ever so slightly to match the movement in the virtual world, they find an increase both in presence and social presence.

In a 2019 poster, Gao and Itoh propose the “Kuroko Paradigm” [5], taking both ideas further. Their concept was to use the potential of Head Mounted Displays (HMD) to hide reality actuators behind avatars in order to potentially improve engagement in AR applications. Their concept used a Wizard of Oz technique, where a human operator took on the role of the reality actuator by using a physical net to play a game of catch the ball. Compared to the Kobito, this system allows for more freedom in the type of reality actuator used, as it should end up completely hidden behind the avatar. In this paper, we will try to automate their system by overlaying the avatar on top of an actual robotic arm engaged in a chess game.

2 SYSTEM DESIGN

Our system is composed of 3 parts: a core Unity¹ project running on a Magic Leap² HMD as well as a Chess engine running in the open-source Lucas Chess GUI³ and Robotic Operating System (ROS) [10] nodes, both running in separate Docker containers. For

*e-mail: fabre@g.ecc.u-tokyo.ac.jp

†e-mail: yuta.itoh@iii.u-tokyo.ac.jp

¹<https://unity.com/>, By Unity Technologies

²<https://www.magicleap.com/magic-leap-2>, By Magic Leap

³<https://lucaschess.pythonanywhere.com>, By Lucas Monge

the robotic arm, we use the UFactory xArm 7⁴ due to its extensive API and wide range of use cases.

2.1 The Core Unity Program

The core of the program uses Unity version 2022.3 and runs directly on the HMD, here a Magic Leap 2. The role of the program is threefold: to act as a link between the Chess engine and the robotic arm, to display the avatar, and to synchronize the robot's coordinates with the chessboard position.

2.1.1 Robot Arm Coordinate Calibration

The program first runs a calibration sequence. Markers placed on the robotic arm and next to the chessboard recognize the initial position of both items in the real world and place their respective 3D models in Unity's coordinate system. For the robotic arm, the marker is placed on its end effector tool. By using the robotic arm URDF file coupled with its current joint positions, we are able to place the base of the robot precisely by running a coordinate transformation from the end effector to the base position. A persistent AR anchor then marks the robot's base coordinates. Saving the anchor allows for faster use of the application on the next usage by skipping the robotic arm calibration, as well as better stability during user movements compared to standard markers. Using the URDF 3D models of the robot, we can also convert from Unity's coordinate system to the robot's fairly easily by matching the scale between the two worlds.

Synchronizing the chessboard position works similarly without the hassle of converting joint coordinates. A marker is placed a fixed distance on the side of the chessboard, which will then instantiate an anchor in its place, in the same vein as the robotic arm calibration. We use a 3D model of the chessboard that is the same size as our real one in order to find the coordinates of each square instantaneously. Once calibrated, the marker can be removed, provided the chessboard remains in place.

2.1.2 Game State Synchronisation

Once the core part is calibrated, the game can start. This hides the robotic arm's 3D model and puts a 3D avatar (here of Unity-Chan⁵) in its place. The 3D avatar makes use of the ML2 Segmented Dimming capabilities to render it opaque and thus hide the robotic arm entirely. In order to make the game as fluid as possible, we use the Chessnut Air electronic chessboard⁶. This allows user to play the game using a normal chessboard while having all their move transcribed instantly on the computer. The Chessnut Air driver and its functionality are available through the chess GUI described in the next section. The core application communicates directly with the chess GUI through a TCP connection. Any move from the computer or the user is instantly communicated to Unity to synchronize the position of each chess piece. When a move from the CPU is detected, the program initiates the robot's move sequence. Each part of the move (piece movement, piece capture, piece promotion, etc.) is decomposed and executed in the correct order. Each piece has a custom height-grabbing point coded in. The robotic arm movement follows the following principle: Move above the target piece; Open Grip; Move downwards, Close Grip; Move Back Up; Move above goal; Move downwards towards goal; Open Grip. To ensure users' safety, we run the robotic arm at a reduced speed, making each move take 7 to 15 seconds to complete. The user is also kept out of the arm's range.

2.2 The Chess GUI

We use the open-source Lucas Chess R2 GUI as a front-end for the chess-related part of the application, including the AI opponent.

⁴<https://www.ufactory.cc/>, By UFactory

⁵<https://unity-chan.com/>, By Unity Technologies Japan G.K.

⁶<https://www.chessnutech.com/>, By Chessnut

Lucas Chess comes with a variety of chess engines, allowing users to play against an opponent matching their skills. We modified the program's source code to include TCP communication capabilities, allowing us to communicate and synchronize the state of the game with the rest of the system.

2.3 The ROS Nodes

To control the robot, we operate on the ROS1 Noetic [10] platform and integrate Unity's ROS TCP Connector⁷ along with the official xArm ROS implementation⁸, which includes interoperability with the "MoveIt" [3] software. The core functionality of the nodes remains unmodified, primarily serving as a communication interface to send commands to the robotic arm.

This modular design keeps the ROS to a minimum, allowing the change of the robotic arm model. The system should work similarly, provided that the new arm is compatible with the "MoveIt" software and/or that the coordinate system conversion part of the system is adapted adequately.

3 DISCUSSIONS AND FUTURE WORK

While the system is simple and re-deployable due to the calibration sequence, much relies on the Chess GUI and electronic chessboard to simplify the design and seamlessly integrate the avatar in the real world. Future works may consider using an AI model to recognize the game state of various board games. Provided that developers can provide an AI opponent as well as a relatively accurate 3D model of the game board for position synchronization, our pipeline could extend to other games such as Go.

The main limitation of our system lies in its fixed position. While the current system can allow for a moving board, this requires a marker to be in view of the user and is, therefore, not desirable. The robotic arm itself is also subject to this limitation. Although it is possible to keep a marker on it due to it being ultimately hidden from view by the avatar, the precision greatly diminishes with distance.

Another limitation lies in the hiding of the robotic arm. While the avatar completely obfuscates the arm most of the time, some side movement makes part of the arm visible. Such a limitation could potentially be addressed by employing Diminished Reality techniques. [8]

Future work on the Kuroko Paradigm should focus on allowing for even more interaction from and with the avatar and evaluating the system's impact on user engagement, presence, and social presence.

4 CONCLUSION

We presented an implementation of the Kuroko Paradigm: Hiding reality actuators behind 3D avatars by superposing a 3D avatar on a robotic arm playing a game of chess. Using only an HMD and a real chessboard, users have the illusion of playing directly against an opponent instead of a computer. We think that this new kind of system will become more commonplace in a future where AI and digital agents become more and more pervasive and encourage further crossing of current robotics technology with AR technology to provide more engaging experiences for users.

ACKNOWLEDGMENTS

This project was partially supported by JST FOREST Grant Number JPMJFR206E.

REFERENCES

- [1] T. Aoki, T. Matsushita, Y. Iio, H. Mitake, T. Toyama, S. Hasegawa, R. Ayukawa, H. Ichikawa, M. Sato, T. Kuriyama, et al. Kobito: virtual brownies. In *ACM SIGGRAPH 2005 emerging technologies*, pp. 11–es. 2005.

⁷<https://github.com/Unity-Technologies/>

ROS-TCP-Connector

⁸https://github.com/xArm-Developer/xarm_ros

- [2] S. Aseeri and V. Interrante. The influence of avatar representation on interpersonal communication in virtual social environments. *IEEE Transactions on Visualization and Computer Graphics*, 2626:1–1, 2021. doi: 10.1109/tvcg.2021.3067783
- [3] D. T. Coleman, I. A. Sucan, S. Chitta, and N. Correll. Reducing the barrier to entry of complex robotic software: a moveit! case study. 2014. doi: 10.6092/JOSER.2014.05_01.P3
- [4] C. Egtebas, G. Klinker, S. Boll, and M. Koelle. Co-speculating on dark scenarios and unintended consequences of a ubiquitous(ly) augmented reality. In *Proceedings of the 2023 ACM Designing Interactive Systems Conference*, DIS '23, p. 2392–2407. Association for Computing Machinery, New York, NY, USA, 2023. doi: 10.1145/3563657.3596073
- [5] T. Gao and Y. Itoh. The kuroko paradigm: The implications of augmenting physical interaction with ar avatars. In *2019 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*, pp. 26–27, 2019. doi: 10.1109/ISMAR-Adjunct.2019.00022
- [6] K. Kim, L. Boelling, S. Haesler, J. Bailenson, G. Bruder, and G. F. Welch. Does a digital assistant need a body? the influence of visual embodiment and social behavior on the perception of intelligent virtual agents in ar. In *2018 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, pp. 105–114, 2018. doi: 10.1109/ISMAR.2018.00039
- [7] M. Lee, K. Kim, S. Daher, A. Raij, R. Schubert, J. Bailenson, and G. Welch. The wobbly table: Increased social presence via subtle incidental movement of a real-virtual table. In *2016 IEEE virtual reality (VR)*, pp. 11–17. IEEE, 2016.
- [8] S. Mori, S. Ikeda, and H. Saito. A survey of diminished reality: Techniques for visually concealing, eliminating, and seeing through real objects. *IPSI Transactions on Computer Vision and Applications*, 9(1):1–14, 2017.
- [9] N. Norouzi, G. Bruder, B. Belna, S. Mutter, D. Turgut, and G. Welch. *A Systematic Review of the Convergence of Augmented Reality, Intelligent Virtual Agents, and the Internet of Things*, pp. 1–24. 2019. doi: 10.1007/978-3-030-04110-6_1
- [10] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng. "ros: an open-source robot operating system". In *"Proc. of the IEEE Intl. Conf. on Robotics and Automation (ICRA) Workshop on Open Source Robotics"*. "Kobe, Japan", may 2009.
- [11] S. Saito, G. Schwartz, T. Simon, J. Li, and G. Nam. Relightable gaussian codec avatars, 2023.
- [12] F. Weidner, G. Boettcher, S. A. Arboleda, C. Diao, L. Sinani, C. Kunert, C. Gerhardt, W. Broll, and A. Raake. A systematic review on the visualization of avatars and agents in ar & vr displayed using head-mounted displays, 5 2023. doi: 10.1109/TVCG.2023.3247072
- [13] W. Zielonka, T. Bagautdinov, S. Saito, M. Zollhöfer, J. Thies, and J. Romero. Drivable 3d gaussian avatars, 2023.